Failure handling (three errors, three reactions) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Where this exercise is saved: `practice/custom_errors_practice.py`, under its `# Failure handling` section — the same file also holds the Basic exercise, under a `# Basic` section. Run it with `cd practice && python custom_errors_practice.py`.

Builds on: the `# Basic` section of that same file — same habit, three error classes instead of one. Nothing to copy or import: scroll up in the file you already have.

Story — `custom_errors_practice.py` (Failure handling section): a real program has more than one way to fail, and each way usually needs a different reaction — retry, ask for new payment info, ask for a new address. This defines three separate exception subclasses and catches each in its own `except` block, so Python's exception matching (which dispatches *by type*) does the sorting for you and the right reaction is wired straight to the right failure. **If not:** a single `except Exception` would treat "out of stock" the same as "payment declined," so the code would end up with an `if/elif` chain re-deriving what the exception type already told it for free.

Solution

```
# practice/custom_errors_practice.py — Failure handling section
class OutOfStockError(Exception):
    pass

class PaymentDeclinedError(Exception):
    pass

class InvalidAddressError(Exception):
    pass

def process_order(problem_type: str) -> None:
    # why: simulates three different real failures on demand, so the
    # except blocks below have something real to react to.
    # when: in real code this would be wherever an order actually gets
    # processed — the type of failure comes from what really went wrong, not a
    # parameter.
    if problem_type == "stock":
```

```
        raise OutOfStockError("item is out of stock")
    if problem_type == "payment":
        raise PaymentDeclinedError("payment was declined")
    if problem_type == "address":
        raise InvalidAddressError("shipping address is invalid")

for problem_type in ["stock", "payment", "address"]:
    try:
        process_order(problem_type)
    except OutOfStockError as e:
        # how: Python checks except blocks top to bottom and stops at the
        # first type match — each error type here gets its own, different
        # reaction.
        print(f"Will retry later, notifying the warehouse: {e}")
    except PaymentDeclinedError as e:
        print(f"Asking the customer for a different payment method: {e}")
    except InvalidAddressError as e:
        print(f"Asking the customer to fix their address: {e}")
```

Expected output:

```
Will retry later, notifying the warehouse: item is out of stock
Asking the customer for a different payment method: payment was declined
Asking the customer to fix their address: shipping address is invalid
```